FORMOSA CRYPTO

Completing the Chain:

Verified Implementations of Hash-Based Signatures and Their Security

Manuel Barbosa François Dupressoir **Rui Fernandes**
Andreas Hülsing Matthias Meijers Pierre-Yves Strub

GitHub Repository

Paper

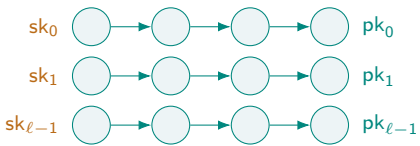
CHES 2026

XMSS: Overview

- XMSS [BDH11] is a stateful hash-based signature scheme.
- Security relies on the security of the underlying hash function.
- XMSS combines a one-time signature scheme (WOTS^+) with a Merkle tree.
  - The root of the Merkle tree is the public key of the XMSS instance.
  - Each leaf corresponds to a (compressed) WOTS^+ public key.
  - A Merkle tree of height h has 2^h leaves, and thus allows signing 2^h messages.

WOTS$^+$: Key Generation

1. Generate ℓ secret n-byte values $\mathsf{sk}_0, \ldots, \mathsf{sk}_{\ell-1}$ (pseudorandomly)
2. Each value starts a hash chain: apply the chaning function $w-1$ steps to obtain
3. The chaining function is defined as $c_k^i(x, \mathbf{r}) = f_k(c_k^{i-1}(x, \mathbf{r}) \oplus r_i)$, with $c_k^0(x, \mathbf{r}) = x$, where $\mathbf{r}$ are randomizers and k is keying parameter for the hash function.



Example: $w = 4$

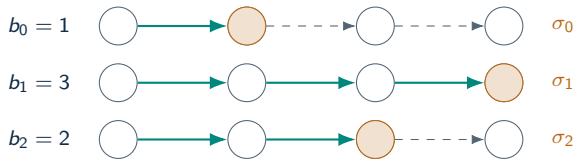
Secret key: $\mathsf{SK} = (\mathsf{sk}_0, \ldots, \mathsf{sk}_{\ell-1})$

Public key: $\mathsf{PK} = (\mathsf{pk}_0, \ldots, \mathsf{pk}_{\ell-1})$

n: element size in bytes; w: Winternitz parameter; ℓ: number of message and checksum digits.

WOTS^+: Signature Generation & Verification

1. Encode the message as base-w digits $m_0, \ldots, m_{\ell_1 - 1}$
2. Append the base-w digits of the checksum $C = \sum_{j=0}^{\ell_1 - 1}(w - 1 - m_j)$, giving ℓ digits $b_0, \ldots, b_{\ell - 1} \in \{0, \ldots, w - 1\}$.
3. The signature is $(\sigma_0, \ldots, \sigma_{\ell - 1})$, where $\sigma_j = c_k^{b_j}(\mathsf{sk}_j, \mathbf{r})$.
4. To verify a signature, it suffices to confirm that completing the chains permits reconstructing the WOTS^+ public key.



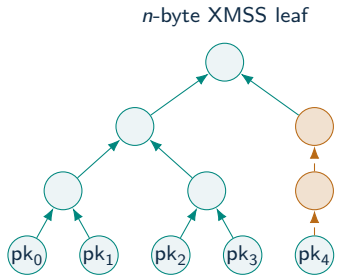
Example: $w = 4$; gold nodes form the signature.

Each WOTS^+ keypair signs at most one message.

L-Tree: Compressing a WOTS$^+$ Public Key

Input: ℓ public-key elements, each n bytes.

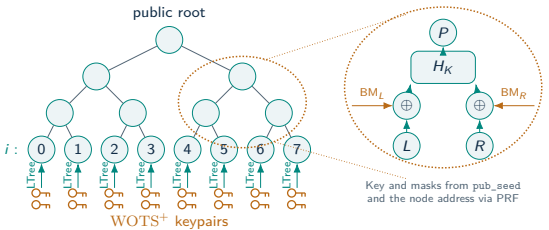
1. Hash adjacent pairs with randomized hashing (hash keys and masks are derived from `pub_seed` and the L-Tree address).
2. If a level has an odd number of nodes, carry the last node to the next level unchanged.
3. Repeat until a single n-byte node remains.



Example: $\ell = 5$
Gold dashed arrows: carry unchanged

XMSS: **Key Generation**

1. Sample three random n-byte seeds:
   - `sk_seed`: PRF seed for WOTS$^+$ secret keys; keeps the secret key compact.
   - `sk_prf`: PRF key for message-hashing randomizers.
   - `pub_seed`: PRF key for randomized-hashing bitmasks.
2. Initialize the signing-key index: $i = 0$.
3. Compute the tree root from the WOTS$^+$ public keys, (compressed into leaves via L-Tree).



$i = 0, \ldots, 2^h - 1$

Example: $h = 3$ (2^h leaves)

Public key: PK $=$ (root, `pub_seed`).

Secret key: SK $=$ (i, `sk_seed`, `sk_prf`, PK).

XMSS^{MT}**:** same process, but compute the root of the top tree in the hypertree; each tree has height h/d.

XMSS: Signature Generation

1. Use the current secret-key index i to select the one-time key; update the stored index to prevent key reuse.
2. Derive the pseudorandom value R using a PRF keyed by `sk_prf`, and compute the message digest using R.
3. Sign the digest with the selected WOTS^+ secret key.
4. Compute the Merkle authentication path for leaf i: it authenticates the corresponding WOTS^+ public key.

Signature

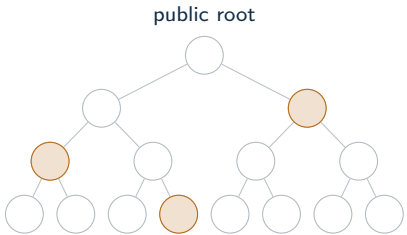
$\sigma = (i, R, \sigma_{\mathrm{WOTS}^+}, \mathsf{auth})$: index, randomizer, WOTS^+ signature, and authentication path.

$\mathrm{XMSS}^{\mathrm{MT}}$: include one WOTS^+ signature and authentication path per layer. For layers $j = 1, \ldots, d-1$, sign the root of the tree below; the initial message digest is computed only once.

XMSS: **Signature Verification**

Inputs: message M, PK $= (\text{root}, \text{SEED})$, $\sigma = (i, R, \sigma_{\text{WOTS}^+}, \text{auth}_i)$, with $0 \leq i < 2^h$.

1. Recompute the message digest $D = H_{\text{msg}}(R \,\|\, \text{root} \,\|\, i, M)$.



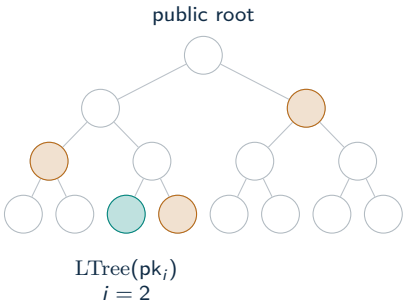
Gold: supplied authentication siblings.

Teal: reconstructed path.

XMSS: **Signature Verification**

Inputs: message M, $\mathsf{PK} = (\mathsf{root}, \mathsf{SEED})$, $\sigma = (i, R, \sigma_{\mathrm{WOTS}^+}, \mathsf{auth}_i)$, with $0 \le i < 2^h$.

1. Recompute the message digest $D = H_{\mathrm{msg}}(R \,\|\, \mathsf{root} \,\|\, i, M)$.
2. Complete the WOTS^+ chains from σ_{WOTS^+} to recover pk_i.



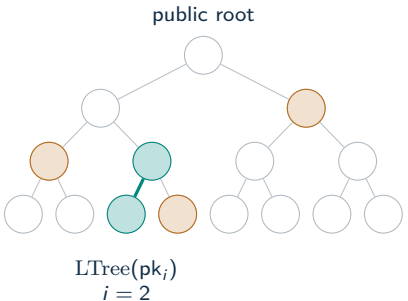
Gold: supplied authentication siblings.

Teal: reconstructed path.

XMSS: **Signature Verification**

Inputs: message M, $\mathsf{PK} = (\mathsf{root}, \mathsf{SEED})$, $\sigma = (i, R, \sigma_{\mathrm{WOTS}^+}, \mathsf{auth}_i)$, with $0 \le i < 2^h$.

1. Recompute the message digest $D = H_{\mathrm{msg}}(R \,\|\, \mathsf{root} \,\|\, i, M)$.
2. Complete the WOTS^+ chains from σ_{WOTS^+} to recover pk_i.
3. LTree compression gives leaf i.



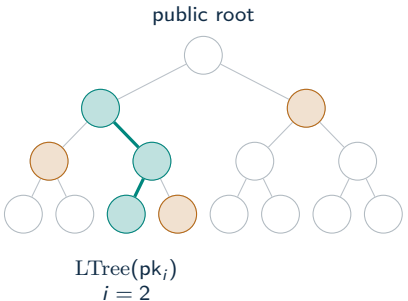
Gold: supplied authentication siblings.

Teal: reconstructed path.

XMSS: **Signature Verification**

Inputs: message M, $\mathsf{PK} = (\mathsf{root}, \mathsf{SEED})$, $\sigma = (i, R, \sigma_{\mathrm{WOTS}^+}, \mathsf{auth}_i)$, with $0 \le i < 2^h$.

1. Recompute the message digest $D = H_{\mathrm{msg}}(R \,\|\, \mathsf{root} \,\|\, i, M)$.
2. Complete the WOTS^+ chains from σ_{WOTS^+} to recover pk_i.
3. LTree compression gives leaf i.
4. Hash upward with the h authentication siblings. Bits of i determine left/right order.



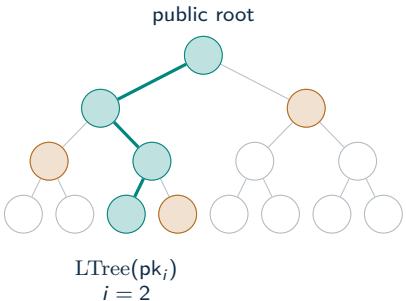
Gold: supplied authentication siblings.

Teal: reconstructed path.

XMSS: **Signature Verification**

Inputs: message M, PK $= (\text{root}, \text{SEED})$, $\sigma = (i, R, \sigma_{\text{WOTS}^+}, \text{auth}_i)$, with $0 \le i < 2^h$.

1. Recompute the message digest $D = H_{\text{msg}}(R \,\|\, \text{root} \,\|\, i, M)$.
2. Complete the WOTS^+ chains from σ_{WOTS^+} to recover pk_i.
3. LTree compression gives leaf i.
4. Hash upward with the h authentication siblings. Bits of i determine left/right order.
5. Accept if the reconstructed root equals root in the public key.

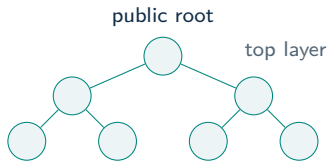


Gold: supplied authentication siblings.

Teal: reconstructed path.

$\mathrm{XMSS}^{\mathrm{MT}}$: Multi-Tree Variant

- $\mathrm{XMSS}^{\mathrm{MT}}$ generalizes XMSS by replacing the single Merkle tree with a hypertree of d layers, each of which contains a Merkle tree of height $\frac{h}{d}$
- Allows the total number of signatures to be increased while keeping running times manageable, at the expense of increased signature size

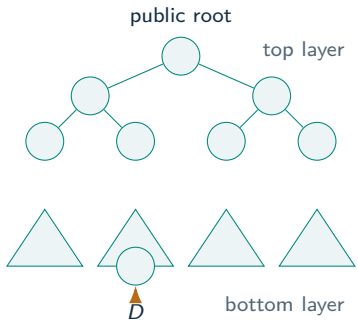


Example: $h = 4$, $d = 2$.

Gold arrows: WOTS^{+} signs a digest or root.

$\mathrm{XMSS}^{\mathrm{MT}}$: Multi-Tree Variant

- $\mathrm{XMSS}^{\mathrm{MT}}$ generalizes XMSS by replacing the single Merkle tree with a hypertree of d layers, each of which contains a Merkle tree of height $\frac{h}{d}$
- Allows the total number of signatures to be increased while keeping running times manageable, at the expense of increased signature size

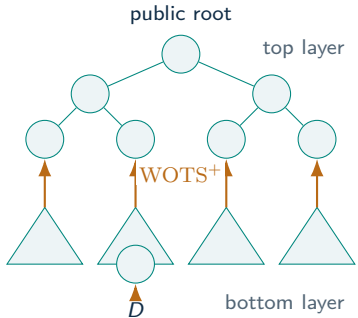


Example: $h = 4$, $d = 2$.

Gold arrows: WOTS^{+} signs a digest or root.

$\mathrm{XMSS}^{\mathrm{MT}}$: Multi-Tree Variant

- $\mathrm{XMSS}^{\mathrm{MT}}$ generalizes XMSS by replacing the single Merkle tree with a hypertree of d layers, each of which contains a Merkle tree of height $\frac{h}{d}$
- Allows the total number of signatures to be increased while keeping running times manageable, at the expense of increased signature size



Example: $h = 4$, $d = 2$.

Gold arrows: WOTS^{+} signs a digest or root.

$\mathrm{XMSS}^{\mathrm{MT}}$: Multi-Tree Variant

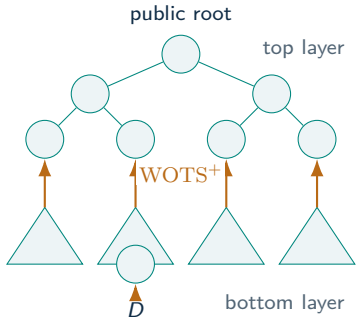
- $\mathrm{XMSS}^{\mathrm{MT}}$ generalizes XMSS by replacing the single Merkle tree with a hypertree of d layers, each of which contains a Merkle tree of height $\frac{h}{d}$
- Allows the total number of signatures to be increased while keeping running times manageable, at the expense of increased signature size

Tradeoff

2^h messages; smaller trees to generate.

Signature: d WOTS^+ signatures and h path nodes.

For $d = 1$, $\mathrm{XMSS}^{\mathrm{MT}}$ is functionally equivalent to XMSS.



Example: $h = 4$, $d = 2$.

Gold arrows: WOTS^+ signs a digest or root.

Jasmin [ABB+17, ABB+20]

- High-level assembly language designed for cryptography implementations
  - High-level zero-cost abstractions: variables, arrays
  - Control over low-level details (e.g. instruction selection, loop unrolling, and whether variables are stored on the stack or in registers)
- Applicability to the development of high-speed, high-assurance cryptographic primitives has been demonstrated through its application to SHA-3 [ABB+19] and ML-KEM [AOB+24, ABB+23].
- Suported architectures: x86_64, ARMv7M, ARMv8-A, RISC-V

Jasmin [ABB+17, ABB+20]

- Stack zeroization [OBG+24]
- (Static) Safety checker
- Compiler correctness proved in Rocq
- Extraction to EasyCrypt
- Speculative constant-time:
  - Spectre v1 [SBG+23] and other spectre variants [ABC+25]
  - Compilation preserves SCT [ABB+25].

Compiler Correctness & Constant-Time Preservation

Compilation preserves the semantics of *safe* source programs and constant-time behavior: functional correctness and constant-time guarantees established at source level carry to the assembly level.

EasyCrypt

- Interactive proof assistant.

Verification Workflow

EasyCrypt

Jasmin

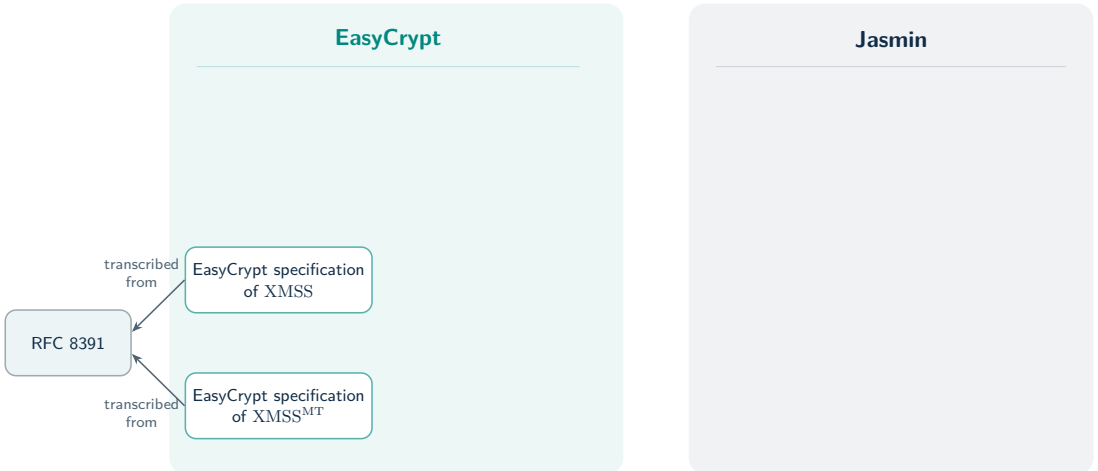
Verification Workflow

RFC 8391

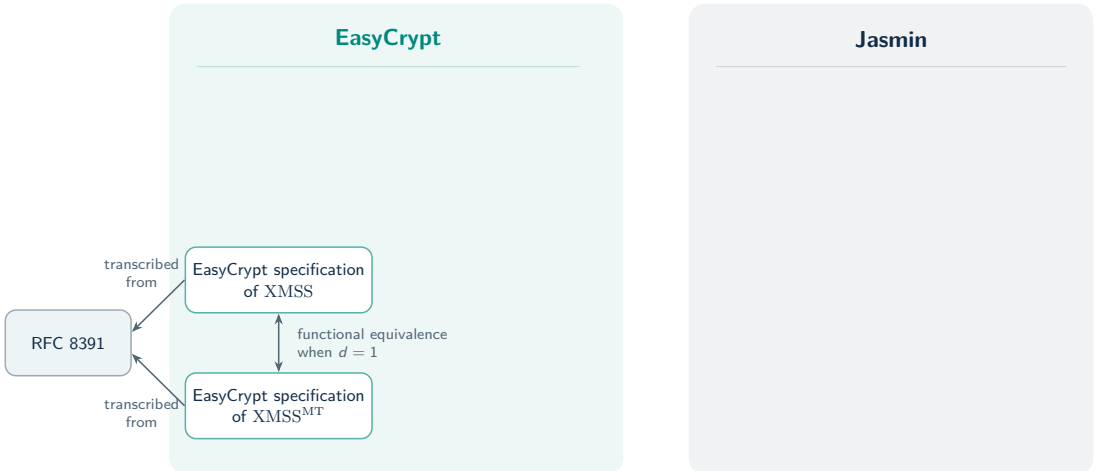
EasyCrypt

Jasmin

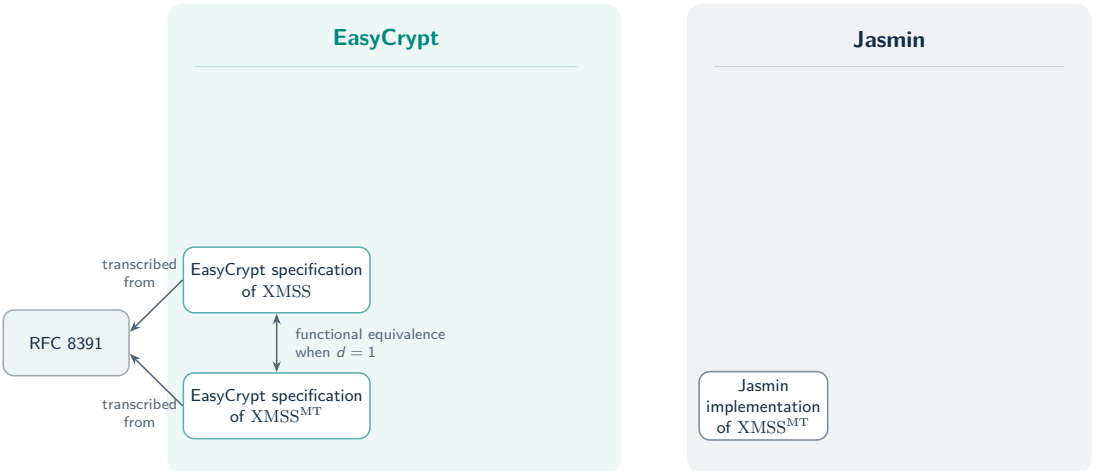
Verification Workflow



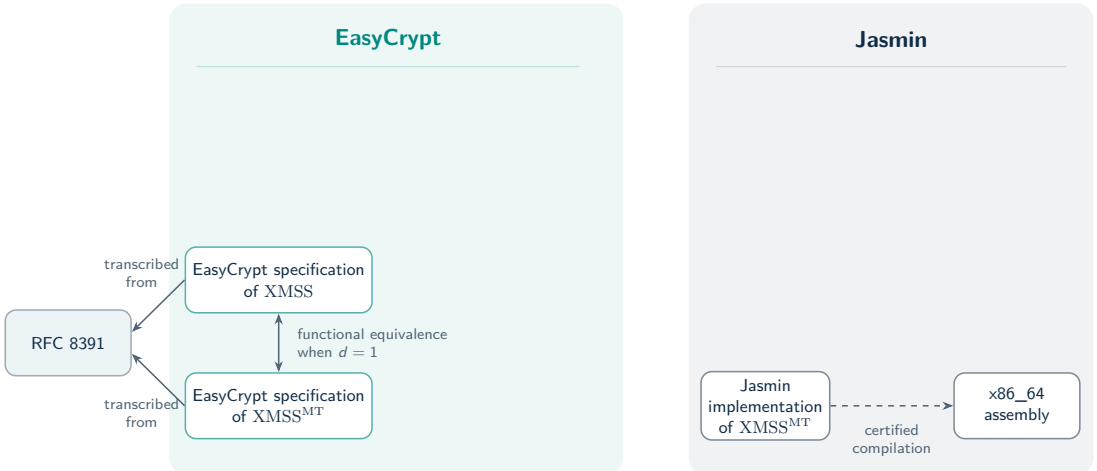
Verification Workflow



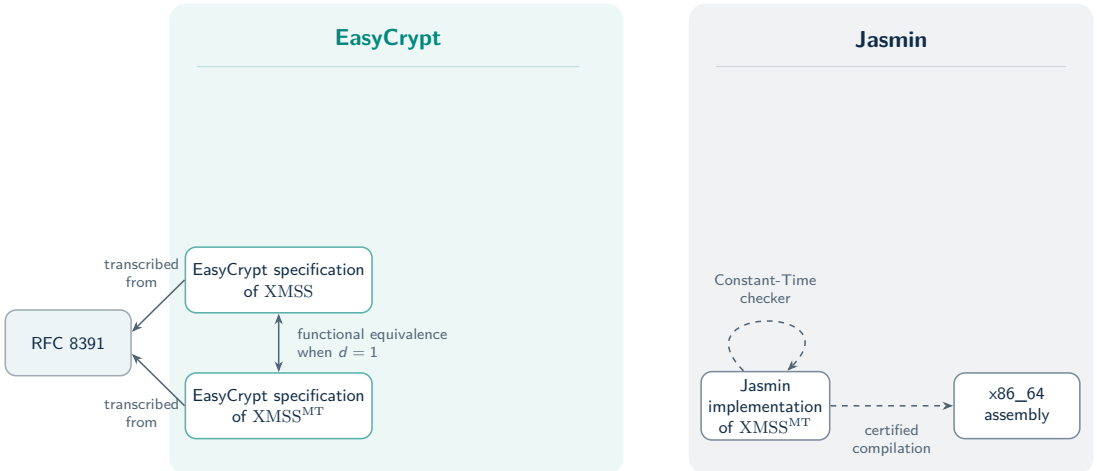
Verification Workflow



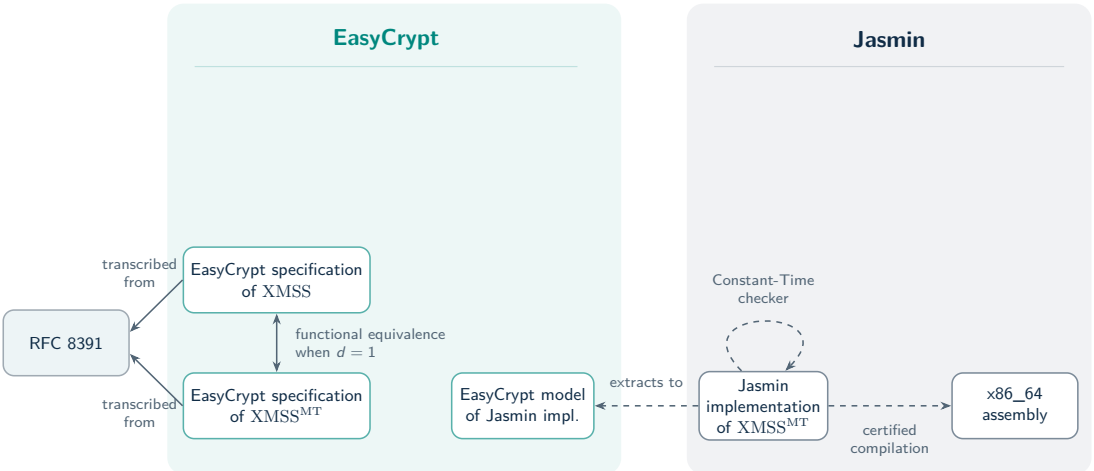
Verification Workflow



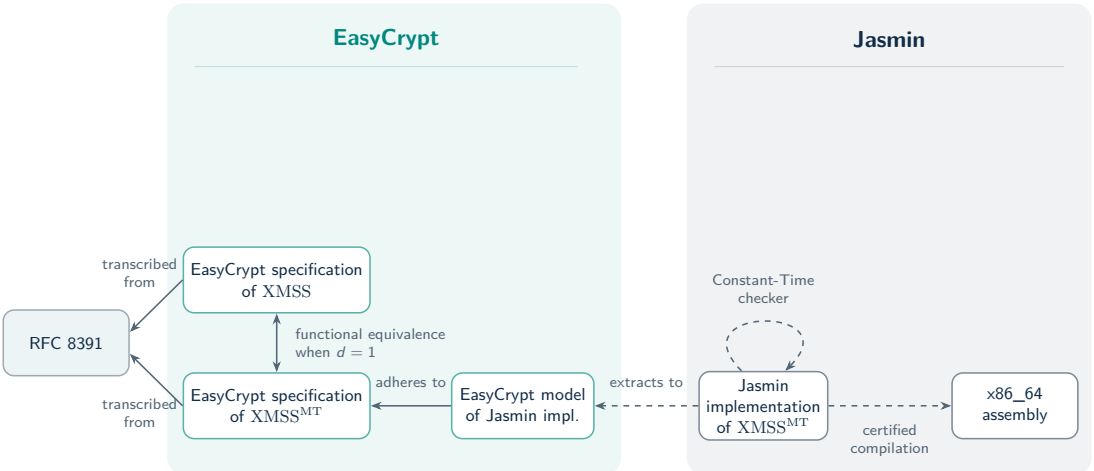
Verification Workflow



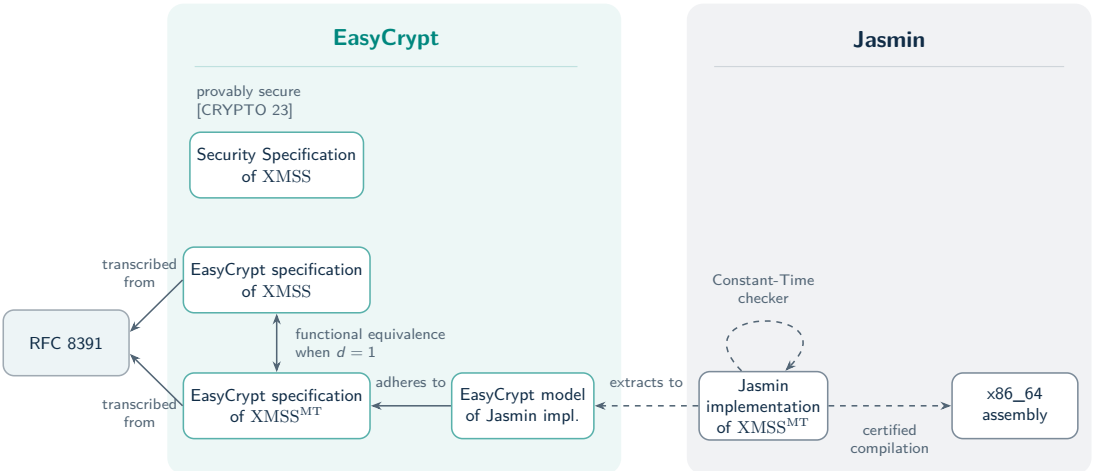
Verification Workflow



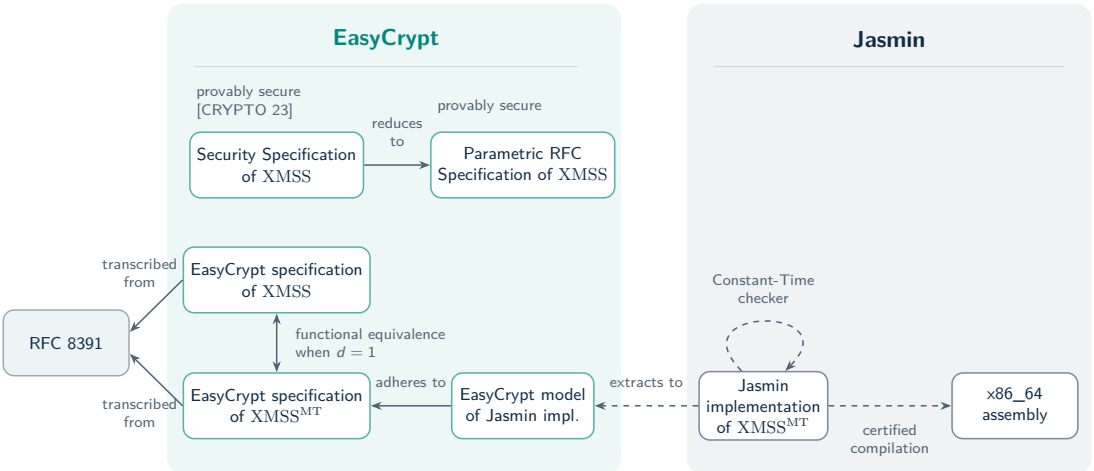
Verification Workflow



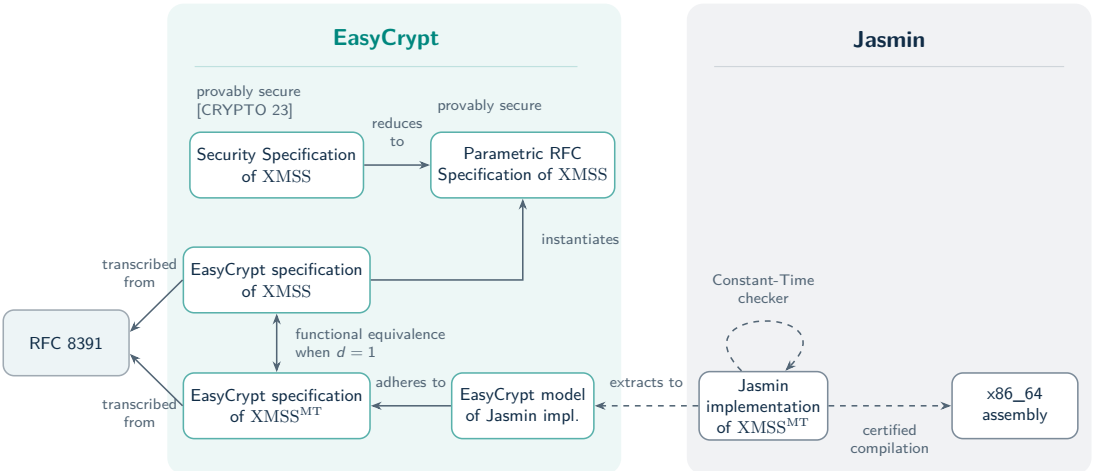
Verification Workflow



Verification Workflow



Verification Workflow



Trusted Computing Base

- Correctness of the SHA-256 implementation is outside the scope of the proof

```
op Hash : W8.t list -> nbytes. (* Abstract operator *)

axiom hash_128 (x : W8.t Array128.t):
  phoare[
    M(Syscall).hash_plen_3n____sha256:
    arg.`2 = x
    ==>
    to_list res = NBytes.val (Hash (to_list x))
  ] = 1%r.
```

Figure: EasyCrypt axiom stating the correctness of SHA-256.

Trusted Computing Base

- Jasmin compiler: unverified OCaml code (parsing and typing)
- Jasmin extraction to EasyCrypt
- EasyCrypt codebase
- SMT Solvers (Z3, CVC4 and Alt-Ergo)

Implementation: Overview

Reference implementation in Jasmin

$\mathrm{XMSS}^{\mathrm{MT}}$ implementation targeting x86_64, based on the C reference implementation[1]. Prioritize readability and closeness to RFC 8391.

- **SHA-256 from libjade**[2]; the C reference uses OpenSSL.
- **Fixed-size data:** stack arrays with sizes known at compile time; a separate hash function for each input length.
- **Variable-size data:** raw pointers with explicit loads and stores, e.g. for the message to sign.
- **Aliasing:** array arguments must be disjoint. The WOTS^{+} chaining function uses a single buffer for in-place updates.

[1] https://github.com/XMSS/xmss-reference

[2] https://github.com/formosa-crypto/libjade

Implementation: Authentication Paths

- **TreeHash:** keep intermediate nodes on a stack; merge the top two nodes whenever they have the same height.
- **Jasmin follows RFC 8391:** compute each authentication-path node with a separate TreeHash call on the corresponding subtree.
- **C reference:** compute the root and authentication path together in a single TreeHash traversal.
- **Tradeoff**: Closer correspondence to the RFC simplifies the correctness proof, but increases execution time.

Implementation: Constant-Time and Zeroization

- **Constant-time validated with Jasmin's checker.** Annotate public values loaded from memory, such as message length: the checker treats memory loads as secret by default.
- **Stack zeroization on return from exported functions:** compiler-generated code clears the stack, ensuring that no secret data remains on the stack after function return.

Measured overhead

Loop zeroization adds less than 0.4% for all three operations. Unrolling adds about 30 KB of code and is slightly slower in these benchmarks.

Implementation: Performance

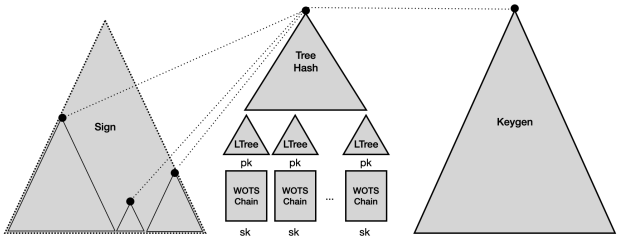
	C Ref	Jasmin (abs, %)	loop zeroization (abs, %)	unrolled zeroization (abs, %)
Keygen	10092.74	8263.54 (-18.12%)	8293.61 (+0.36%)	8312.35 (+0.59%)
Sign	20251.01	24775.17 (+22.34%)	24872.05 (+0.39%)	24927.53 (+0.61%)
Verify	10.35	8.47 (-18.14%)	8.48 (+0.13%)	8.95 (+5.69%)

Table: Performance Comparison between C and Jasmin implementations in 10^6 clock cycles (column #2). Zeroization overhead w.r.t. Jasmin baseline (columns #3 and #4).

- **Keygen and verify:** about 18% fewer cycles, driven by the faster SHA-2 implementation.
- **Sign:** about 22% more cycles, due to separate TreeHash calls for authentication-path nodes.

TreeHash

- Computes the root of a Merkle subtree from its leaves.
- **Security Specification**: Expressed as a recursive function over an inductive data type for trees.
- **RFC Specification**: Iterative function that makes use of a stack for storing intermediate nodes.



Functional Correctness: Proof Approach

Goal

Prove that the Jasmin implementation of $\mathrm{XMSS}^{\mathrm{MT}}$ agrees with the RFC-like EasyCrypt specification from Section 3 of the paper.

- Extract an EasyCrypt representation of the Jasmin program using the Jasmin compiler.
- Prove **pRHL equivalence lemmas** relating the extracted code to the specification, under matching parameters and inputs.
- Show that both sides make consistent calls to the underlying hash function; assume correctness of the SHA-2 implementation.

Functional Correctness: Relating Representations

- **Specification:** records for keys and signatures.
- **Implementation:** byte arrays and memory buffers.
- Encoding and decoding operators connect these representations

What the equivalence lemmas establish

Key generation: implementation key buffers correspond to the specification's public and secret keys.

Verification: the implementation returns a 64-bit zero *if and only if* the specification accepts the signature.

Functional Correctness: Reasoning About Memory

- Raw pointers require explicit reasoning about address arithmetic and the contents of memory.
- **`valid_ptr_i`:** adding an accessed non-negative offset does not overflow the 64-bit address space.
- **`disjoint_ptr`:** the specified memory regions do not overlap.

Example: signature verification

The signed-message input, recovered-message output, and output-length buffer are assumed disjoint. Writes to the outputs cannot modify input data or interfere with each other.

Functional Correctness: TreeHash

- **Most demanding proof:** relate the intermediate stacks and loop state of the implementation and specification.
- The relational loop invariant establishes **identical stack accesses** and matching results, avoiding a second proof of stack bounds at this stage.
- **Safety checked separately:** Jasmin's safety checker guarantees that the implementation performs no out-of-bounds access.
- Stack accesses in the RFC specification are proved within bounds later, when connecting it to the security proof.

Proof structure

The equivalence proof establishes matching computation; the separate safety check establishes safe memory accesses.

QUESTIONS?

References I

[ABB+17] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt, and Pierre-Yves Strub.

Jasmin: High-assurance and high-speed cryptography.

In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1807–1823. ACM Press, October / November 2017.

[ABB+19] José Bacelar Almeida, Cecile Baritel-Ruet, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Alley Stoughton, and Pierre-Yves Strub.

Machine-checked proofs for cryptographic standards: Indifferentiability of Sponge and secure high-assurance implementations of SHA-3.

In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 1607–1622. ACM Press, November 2019.

References II

[ABB+20] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Adrien Koutsos, Vincent Laporte, Tiago Oliveira, and Pierre-Yves Strub.

The last mile: High-assurance and high-speed cryptographic implementations.

In *2020 IEEE Symposium on Security and Privacy*, pages 965–982. IEEE Computer Society Press, May 2020.

[ABB+23] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Tiago Oliveira, Hugo Pacheco, Miguel Quaresma, Peter Schwabe, Antoine Séré, and Pierre-Yves Strub.

Formally verifying Kyber episode IV: Implementation correctness.

IACR TCHES, 2023(3):164–193, 2023.

[ABB+25] Santiago Arranz-Olmos, Gilles Barthe, Lionel Blatter, Benjamin Grégoire, and Vincent Laporte.

Preservation of speculative constant-time by compilation.

Proc. ACM Program. Lang., 9(POPL):1293–1325, 2025.

References III

[ABC+25] Santiago Arranz-Olmos, Gilles Barthe, Chitchanok Chuengsatiansup, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Peter Schwabe, Yuval Yarom, and Zhiyuan Zhang.

Protecting cryptographic code against spectre-rsb: (and, in fact, all known spectre variants).

In Lieven Eeckhout, Georgios Smaragdakis, Katai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach, editors, *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, pages 933–948. ACM, 2025.

[AOB+24] José Bacelar Almeida, Santiago Arranz Olmos, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Cameron Low, Tiago Oliveira, Hugo Pacheco, Miguel Quaresma, Peter Schwabe, and Pierre-Yves Strub.

Formally verifying Kyber - episode V: Machine-checked IND-CCA security and correctness of ML-KEM in EasyCrypt.

In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part II*, volume 14921 of *LNCS*, pages 384–421. Springer, Cham, August 2024.

References IV

[BDH11] Johannes Buchmann, Erik Dahmen, and Andreas Hülsing.
XMSS - A practical forward secure signature scheme based on minimal security assumptions.
In Bo-Yin Yang, editor, *Post-Quantum Cryptography - 4th International Workshop, PQCrypto 2011, Taipei, Taiwan, November 29 - December 2, 2011. Proceedings*, volume 7071 of *Lecture Notes in Computer Science*, pages 117–129. Springer, 2011.

[OBG+24] Santiago Arranz Olmos, Gilles Barthe, Ruben Gonzalez, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Tiago Oliveira, and Peter Schwabe.
High-assurance zeroization.
IACR TCHES, 2024(1):375–397, 2024.

[SBG+23] Basavesh Ammanaghatta Shivakumar, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Swarn Priya, Peter Schwabe, and Lucas Tabary-Maujean.
Typing high-speed cryptography against spectre v1.
In *2023 IEEE Symposium on Security and Privacy*, pages 1094–1111. IEEE Computer Society Press, May 2023.